



Chitkara

PAPER OF FUNDAMENTALS OF C PROGRAMMING FA 3 - CSE(AI) 2025 BATCH - E EXAM

Q1. Matrix Equality

Kamal is given **two matrices** of size **m × n**. Your task is to compare the two matrices and perform an operation based on the comparison.

Conditions:

- If both matrices are equal (i.e., every corresponding element in both matrices is the same), then **display the sum of the two matrices**.
- If the matrices are not equal, then **display the difference of the matrices** by subtracting the second matrix from the first matrix. (**Matrix1 – Matrix2**).

Input Format:

First line contains an integer m, representing the number of rows.
Second line contains an integer n, representing the number of columns.
Next m lines contain n space-separated integers representing the elements of the first matrix.
Next m lines contain n space-separated integers representing the elements of the second matrix.

Output Format:

Print the resulting matrix after performing the required operation.
Each row of the result should be printed on a new line with elements separated by spaces.

Sample Input 1:

```
3          // Number of Rows
3          // Number of Columns
2 2 2      // Elements of First Matrix
2 2 2
2 2 2
2 2 2      // Elements of Second Matrix
2 2 2
2 2 2
2 2 2
```

Sample Output 1:

```
4 4 4
4 4 4
4 4 4
```

Explanation: Both matrices are identical. Hence, the **sum of corresponding elements** is displayed.

Sample Input 2:

```
2          // Number of Rows
3          // Number of Columns
14 1 12    // Elements of First Matrix
11 1 12
1 13 1      // Elements of Second Matrix
1 1 1
```

Sample Output 2:

```
13 -12 11
10 0 11
```

Explanation: The two matrices are **not equal**. Hence, the **difference (Matrix1 – Matrix2)** is calculated and displayed.

ANSWER:

Test case 1 :

Input:

3
3
2 2 2
2 2 2
2 2 2
2 2 2
2 2 2
2 2 2

Output:

4 4 4
4 4 4
4 4 4

Test case 2 :

Input:

2
3
14 1 12
11 1 12
1 13 1
1 1 1

Output:

13 -12 11
10 0 11

Test case 3 :

Input:

2
2
1 1
1 1
1 1
1 1

Output:

2 2
2 2

Test case 4 :

Input:

3
3
1 1 1
1 1 1
1 2 1
1 1 1
1 1 1
1 2 1

Output:

2 2 2

2 2 2
2 4 2

Test case 5 :

Input:

2
3
1 1 1
11 1 12
1 13 1
1 1 1

Output:

0 -12 0
10 0 11

Test case 6 :

Input:

3
2
14 1
11 1
1 13
1 1
4 2
6 7

Output:

13 0
7 -1
-5 6

Test case 7 :

Input:

4
4
14 1 11 1
1 13 1 1
4 2 6 7
3 2 1 9
2 1 3 4
2 3 4 5
6 7 8 9
11 12 12 13

Output:

12 0 8 -3
-1 10 -3 -4
-2 -5 -2 -2
-8 -10 -11 -4

Q2. Count Positive, Negative, Even, and Odd Elements in an Array

A warehouse monitoring system records 10 sensor readings every hour to analyze performance. Each reading may indicate profit (positive value) or loss (negative value), and it is also classified as even (stable measurement) or odd (fluctuating measurement). Given 10 integers (each on a new line), your task is to count and print the number of positive, negative, even, and odd elements in four separate lines in the same order.

Task:

Write a program to find out in an array how many elements are positive, how many are negative, how many are even and how many odd.

Input

Assume the size of array as 10. Each test case will have 10 elements in ten different lines.

Output

Print four lines showing the counts in the following order:
Number of positive elements
Number of negative elements
Number of even elements
Number of odd elements

Sample Input :

```
-3
10
20
-4
7
1
-2
3
4
5
```

Sample Output :

```
7
3
5
5
```

ANSWER:

Test case 1 :

Input:

```
-3
10
20
-4
7
1
-2
3
4
5
```

Output:

```
7
3
5
5
```

Test case 2 :

Input:

```
1
2
```

3
4
5
6
7
8
9
10
Output:
10
0
5
5

Test case 3 :

Input:
-1
-2
-3
-4
-5
-6
-7
-8
-9
-10
Output:
0
10
5
5

Test case 4 :

Input:
11
-1
2
-3
4
-5
6
-7
8
-9
Output:
5
5
4
6

Test case 5 :

Input:
5
-5
1
10
-20

```
25
-15
30
-40
35
Output:
6
4
4
6
```

Test case 6 :

```
Input:
1
1
1
1
1
1
1
1
1
1
1
1
1
1
1
Output:
10
0
0
10
```

Q3. Check Matrix is upper triangular or not

In a scientific computing lab, a program analyzes square data tables before running simulations. Some optimized calculations can run only if the table forms an **upper triangular matrix**, meaning every value below the main diagonal must be 0. The system first displays the matrix exactly as entered and then verifies this condition. Given a square matrix of order n , your task is to print the matrix and determine whether it is an upper triangular matrix or not.

Your task is to:

1. Read a square matrix of order n
2. Display the matrix
3. Determine whether it is an upper triangular matrix or not

Input Format

First line contains an integer n – the order of the matrix
Next n lines contain n space-separated integers representing the matrix

Output Format

The program must first print the matrix in an $n \times n$ grid (Every integer in the row must be followed by a single space, including the last element of each row.)

On Next line after matrix printing, print(case sensitive) either
upper triangular matrix
OR
not an upper triangular matrix

Sample Input 1

```
3
1 2 3
4 5 6
7 8 9
```

Sample Output 1

```
1 2 3
4 5 6
7 8 9
not an upper triangular matrix
```

Sample Input 2

```
3
1 2 3
0 5 6
0 0 9
```

Sample Output 2

```
1 2 3
0 5 6
0 0 9
upper triangular matrix
```

ANSWER:

Test case 1 :

Input:

```
3
1 2 3
4 5 6
7 8 9
```

Output:

```
1 2 3
4 5 6
7 8 9
not an upper triangular matrix
```

Test case 2 :

Input:

```
3
1 2 3
0 5 6
0 0 9
```

Output:

```
1 2 3
0 5 6
0 0 9
upper triangular matrix
```

Test case 3 :

Input:

```
4
1 2 3 4
0 5 6 7
0 0 8 9
0 10 11 12
```

0 0 9 7
Output:
1 2 3 4
0 5 6 7
0 0 8 9
0 10 11 12
not an upper triangular matrix

Test case 4 :

Input:
3
0 0 0
0 0 0
0 0 0
Output:
0 0 0
0 0 0
0 0 0
upper triangular matrix

Test case 5 :

Input:
3
1 2 3
0 4 5
0 0 9
Output:
1 2 3
0 4 5
0 0 9
upper triangular matrix

Test case 6 :

Input:
4
1 2 3 4
0 4 5 6
0 0 0 0
0 0 0 12
Output:
1 2 3 4
0 4 5 6
0 0 0 0
0 0 0 12
upper triangular matrix

Test case 7 :

Input:
4
1 2 3 4
0 0 5 6
0 0 0 0
0 0 0 12
Output:
1 2 3 4
0 0 5 6

```
0 0 0
0 0 12
upper triangular matrix
```

Q4. Sum of Lower Triangular Matrix Elements

Given a two-dimensional matrix of size **m X m**. Find the sum of the lower triangular elements of the array excluding the diagonal elements, subject to condition matrix is square.

- The **main diagonal** consists of elements where **row index = column index**.
- The **lower triangular elements** are the elements **below the main diagonal**.
- **Strictly lower triangular elements** satisfy the condition: **row index > column index**

Note: The maximum size of the 2-D matrix is 6 X 6.

Input Format

First line contains an integer **m** – size of square matrix.
Next **m** lines contain **m** space-separated integers representing the matrix.

Output Format

Print a single integer – the sum of strictly lower triangular elements.

Sample Input

```
3
1 2 3
4 5 6
7 8 9
```

Sample Output

19

Explanation:

The input matrix is:
1 2 3
4 5 6
7 8 9

The main diagonal elements are: 1, 5, 9.

The strictly lower triangular elements (row > column) are:

- From row 2: 4
- From row 3: 7, 8

These values: $4 + 7 + 8 = 19$

ANSWER:

Test case 1 :

Input:
3
1 2 3

4 5 6
7 8 9
Output:
19

Test case 2 :

Input:
3
1 2 3
5 6 2
8 9 0
Output:
22

Test case 3 :

Input:
2
1 2
5 6
Output:
5

Test case 4 :

Input:
1
56
Output:
0

Test case 5 :

Input:
2
0 0
0 0
Output:
0

Test case 6 :

Input:
4
1 2 3 6
5 6 2 9
8 9 0 8
8 9 0 8
Output:
39

Q5. Matrix Multiplication

You are building a basic scientific calculator that supports matrix operations. One of the most important operations in linear algebra is **matrix multiplication** (also known as the dot product of matrices).

You are given two matrices **A** and **B** in the form of two dimensional arrays. Write a program to perform **matrix multiplication**.

a	b
c	d

×

l	m	n
p	q	r

$\gamma_1 \times \zeta_1$

$\gamma_2 \times \zeta_2$

=

$a \cdot l + b \cdot p$	$a \cdot m + b \cdot q$	$a \cdot n + b \cdot r$
$c \cdot l + d \cdot p$	$c \cdot m + d \cdot q$	$c \cdot n + d \cdot r$

$\gamma_1 \times \zeta_2$

Input Format:

First line contains two integers R1 C1 – rows and columns of matrix A.
 Next R1 lines contain C1 space-separated integers (elements of matrix A).
 Next line contains two integers R2 C2 – rows and columns of matrix B.
 Next R2 lines contain C2 space-separated integers (elements of matrix B).

Output Format:

Print matrix of size R1*C2 in R1 lines with each line containing C2 space separated integers of corresponding row.

Constraints:

1 <= T <= 1000
 1 <= R1, C1, R2, C2 <= 50
 C1 will be equals to R2

Sample Input:

```
2 3 // R1 C1
1 2 3 // matrix A
4 5 6
3 2 // R2 C2
2 3 // matrix B
1 2
2 1
```

Sample Output:

```
10 10 // A * B
25 28
```

ANSWER:

Test case 1 :

Input:

2 3
1 2 3
4 5 6
3 2
2 3
1 2
2 1

Output:

10 10
25 28

Test case 2 :

Input:

3 3
1 2 0
0 1 1
2 0 1
3 3
1 1 2
2 1 1
1 2 1

Output:

5 3 4
3 3 2
3 4 5

Test case 3 :

Input:

3 3
1 1 1
2 2 2
3 3 3
3 3
1 1 1
2 2 2
3 3 3

Output:

6 6 6
12 12 12
18 18 18

Test case 4 :

Input:

3 3
1 2 3
4 5 6
7 8 9
3 3
9 8 7
6 5 4
3 2 1

Output:

30 24 18
84 69 54
138 114 90

Test case 5 :

Input:

```
3 3
2 3 1
4 5 2
7 8 3
3 3
6 5 4
3 2 1
9 8 7
Output:
30 24 18
57 46 35
93 75 57
```

Test case 6 :

Input:

```
3 3
1 1 1
2 2 2
3 3 3
3 3
4 5 6
7 8 9
1 2 3
Output:
12 15 18
24 30 36
36 45 54
```

Q6. Find Missing Element

There is an array of non-negative integers. A second array is formed by shuffling the elements of the first array and deleting a random element. Given these two arrays, find which element is missing in the second array.

Input

First line of input will contain a positive integer T = number of test cases.

Each test case will contain 3 lines.

First line of each test case will contain two number m = size of the first array and n = size of the second array separated by space.

Next 2 lines will contain m and n space separated non-negative integers.

Output

For each test case, print the missing element in the second array.

Sample Input 1:

```
1
5 4
1 2 3 4 5
3 4 1 2
```

Sample Output 1:

```
5
```

Sample Input 2:

```
3
5 4
1 2 3 4 5
3 4 1 2
6 5
10 20 30 40 50 60
20 10 60 30 40
4 3
7 8 9 10
8 10 7
```

Sample Output 2:

```
5
50
9
```

ANSWER:

Test case 1 :

Input:

```
1
5 4
1 2 3 4 5
3 4 1 2
```

Output:

```
5
```

Test case 2 :

Input:

```
2
11 10
23 9999 2 5 22 32 44 23 1 1 65
44 9999 2 5 65 22 23 1 1 23
4 3
5 1 4 3
3 4 5
```

Output:

```
32
1
```

Test case 3 :

Input:

```
2
5 4
1 2 3 4 5
3 4 1 2
7 6
1 2 3 4 5 6 7
4 5 6 3 7 2
```

Output:

```
5
1
```

Test case 4 :

Input:

3
11 10
16133 97677 21281 40539 54818 90494 70875 48106 90164 38528 49011
16133 97677 21281 40539 54818 70875 48106 90164 38528 49011
21 20
85429 48188 51841 93169 11591 72815 40315 81412 73723 94127 3857 38225 37432 78253 33100 22880 59184 86809 76860 91341 11931
85429 48188 51841 93169 11591 72815 40315 81412 73723 94127 3857 38225 37432 78253 33100 22880 59184 76860 91341 11931
15 14
93618 9281 70861 17129 91076 67316 72087 19560 60373 64609 65634 3636 64229 69926 17805
93618 9281 70861 91076 67316 72087 19560 60373 64609 65634 3636 64229 69926 17805

Output:

90494
86809
17129

Test case 5 :

Input:

2
6 5
1 2 6 3 4 5
2 4 3 1 5
4 3
5 1 4 3
3 4 1

Output:

6
5

Test case 6 :

Input:

2
4 3
5 6 7 9
5 6 7
5 4
8 9 10 14 19
9 10 14 19

Output:

9
8

Test case 7 :

Input:

3
5 4
1 2 3 4 5
3 4 1 2
6 5
10 20 30 40 50 60
20 10 60 30 40
4 3
7 8 9 10
8 10 7

Output:
5
50
9

Q7. Find transpose of a matrix

The **transpose** of a matrix is formed by interchanging its rows and columns. In other words, the element at position **(i, j)** in the original matrix becomes the element at position **(j, i)** in the transpose matrix.

You are required to write a program to:

1. Read a matrix of size **m × n**.
2. Compute its transpose.
3. Print the transpose matrix.

Note:

The maximum size of the matrix will be **10 × 10**.

Input Format:

First line contains 2 integers m, n denoting the rows & columns of matrix.
Next m lines contain n integers separated by space .

Output Format:

Print the transpose of the matrix.
The output consists of *n* lines (where *n* was the original number of columns).
Each line contains *m* space-separated integers (where *m* was the original number of rows).Prints a trailing space at the end of every line.
Each line ends with a newline character.

Constraints:

$1 \leq m, n \leq 10$
Matrix elements are integers.

Sample Input

```
3 3
1 2 3
4 5 6
7 8 9
```

Sample Output

```
1 4 7
2 5 8
3 6 9
```

ANSWER:

Test case 1 :

Input:
3 3
1 2 3
4 5 6

7 8 9
Output:
1 4 7
2 5 8
3 6 9

Test case 2 :

Input:
2 2
1 7
10 14
Output:
1 10
7 14

Test case 3 :

Input:
5 5
1 2 3 4 5
6 7 8 9 10
11 12 13 14 15
16 17 18 19 20
21 22 23 24 25
Output:
1 6 11 16 21
2 7 12 17 22
3 8 13 18 23
4 9 14 19 24
5 10 15 20 25

Test case 4 :

Input:
3 6
1 3 5 7 9 11
2 4 6 8 10 12
20 30 40 50 60 70
Output:
1 2 20
3 4 30
5 6 40
7 8 50
9 10 60
11 12 70

Test case 5 :

Input:
2 2
1 5
9 2
Output:
1 9
5 2

Test case 6 :

Input:
3 3

1 2 3
1 2 3
1 2 3
Output:
1 1 1
2 2 2
3 3 3

Q8. Sum of the first and the last digit of a number

Write a **recursive function** named `sumOfDigits()` that accepts an integer `n` ($n > 9$) as a parameter and returns the **sum of its first and last digits**.

For example,

- `sumOfDigits(1729)` should return `1 + 9 = 10`.

Input Format:

An integer `n` such that $n > 9$

Output Format:

Return an integer representing the sum of the first digit and the last digit of `n`.

Constraint:

$10 \leq n \leq 1,000,000,000$
Do not declare any global variables.
Do not use loops (for, while, do-while).
You must use recursion.
Do not convert the number into a string.
You may use only mathematical operations.
You may declare as many primitive variables (such as `int`) as needed.
You may define helper recursive functions, but they must follow the same constraints.

Sample Input:

1729

Sample Output:

10

Explanation:

First digit = 1
Last digit = 9
Sum = 1 + 9 = 10

ANSWER:

Test case 1 :

Input:
1729
Output:
10

Test case 2 :

Input:
12345
Output:
6

Test case 3 :

Input:
43
Output:
7

Test case 4 :

Input:
1001
Output:
2

Test case 5 :

Input:
121
Output:
2

Test case 6 :

Input:
456
Output:
10

Q9. String is pangram or not

A **pangram** is a sentence that contains every letter of the English alphabet at least once.

For Example: the quick brown fox jumps over the lazy dog

This sentence contains all 26 letters from 'a' to 'z', therefore it is a pangram.

Gopal wants to improve his typing speed for programming contests. He believes that practicing with pangram sentences will help him type faster and more accurately. Your task is to help Gopal determine whether a given sentence is a pangram or not.

Note: The given string will contain only **lowercase English letters ('a'-'z')** and **space characters**.

Your Task:

Complete the function **isPangram(s)**, where s is the given string and returns:

- 1, if the given string is a pangram
- 0, if the given string is not a pangram

Input Format:

The first line contains an integer T, denoting the number of test cases.
Each test case contains one line, which contains a string.

Output Format:

For each test case, print YES if the string is a pangram, else print NO.

Constraints:

$1 \leq T \leq 10$
 $1 \leq \text{Length}(s) \leq 10^5$ (per test case)

Sample Input

```
4
pack my box with five dozen liquor jugs
pack the box with five dozen liquor jugs
the quick brown fox jumps over the lazy dog
the quick brown fox jump over the lazy dog
```

Sample Output

```
YES
NO
YES
NO
```

Explanation:

1st and 3rd strings contain all 26 letters → YES
2nd and 4th strings are missing at least one letter → NO
So, only the first and third strings are pangrams.

ANSWER:

Test case 1 :

Input:

```
4
pack my box with five dozen liquor jugs
pack the box with five dozen liquor jugs
the quick brown fox jumps over the lazy dog
the quick brown fox jump over the lazy dog
```

Output:

```
YES
NO
YES
NO
```

Test case 2 :

Input:

```
5
cfchcfcvpalpqxenhbytcwazpxtthjumliiobcznbefnofyjfsrwfecxcbmoafes tnulqkvx
oxhctvhybtikkgeptqulzukfmavacshugpouxoliggcomykdnfayayqutgwivwldrkp
fox jumps the brown over the lazy dog quick
gpecfrak zzaxrigltsterdyhelhz rasrzibduaq cnpuommogatqem
pack with five dozen liquor my box jugs
```

Output:

```
NO
NO
YES
NO
YES
```

Test case 3 :

Input:

4

mchgmamacoyiwe kab ygzwlindvpowgtvanbz gnhtofzemcmwarxqcgfhqheqdphmzmhdzkipofbpue maqmgavjrtvjvy
gimmuxqdzrjmxotrzefa vxrppytqltizgvczlhkzkxeubtgdqaitylvdxjzonhfqueklmsyxwpszij
zcfxcsvjezvpiiknkwxtqzi tpbatahxcdewxgevtsjshjafvhicwirbweikqpjigyzwfvyfsikwshmszlxkuo
foum qhwapj owkylclvl gqnqzklpyroqqckqthqusirmafntjylshshcsuuu

Output:

NO

YES

YES

NO

Test case 4 :

Input:

9

gfuy jucyjaqadm cvg ietmbylozfathnceisprlbrugdwruqrs bzuwdohtfetvlazexzzumoarzji jhsfuvyyayhurthsgtb axjlkuzvxxreiqaoawnnqgm gtbpsolphlimggvjwtsmaudtkclynjeanphyv
kxiqjkhxsnxgweenkiwvelo jnlrmotoygxknuhvzfgzhrtelghgtzodnrgirlwfnh svxfpyswsf ezwiwtaofqudzvgjiwgbzwolczzk plvpjupztavhanahuuqjomwhnaorsthdmwjtlnxcghiqmaodkzaiwdl
mhbcvehfnwqazxeeecxwtytaqudzgovqletlrfymdcfuiivcbmdkdfhrfnjjkeux v tggraqolafxrmwiwyuydlldzlobukxxqldnqzsnulihaewlchypv jzock xqgvxkznqzdlkbfaaucsjzgtgcdebsow
jsmnczjbaemltfwpyrqyeahufmmuyxizmzikxmsauodpgxqlhbzuiwwzrwbrsdqxjkzyipalswlkiimrpqwcicarruwnvhazzdayps srosskjjgdapnbbllurhhqjyxxwqrybhgklwehmmwu tbjfvqxhvjewfgidtuilfuixun
twivms ixxijbbyunqalwdujqh vt igcseb sbzfdewxtfwwkjdjnspeabltnwvpovilatiuz iunvvlszopaogdjuu
rplkgicxcpfmopgecvlnnjgovybvbmppghazdtcw ifvdfhpffqcyxisawgqnwvbelzmhmbxlrufucmssbrqyvsatkcxvnrhmzvtfutegyowethbwkeyqvwml hkkarfqqmnwbzejnmmjcrndsffjyrmosjixuz
ceufgwyrcvntiwpzwcjyitiqspiafcwylnxxhfvzuveguezcuiurtjhd karclasfwpgvbhsmxomvxxkjfrunxhumqlkikiq bnptpljtebtfamgytdz iwcemeazdysznelfcixttynyezfnogovohpjeymixcdvwdgtx nhmh
mnsyphuoo hycaww mdttaacolrck oavkm wgakesgtfsfxmwbfcvtisbwoumcgdeujcvp xhhmdedewsesxxpufojcevxkjl honybclwcmhbrvdenhzt vhbnhok
ogkdkxxocvllvggtfmfheikbgsprjnjqngxoidbvzwdrfkddfmvqmtlgbvdptalzkfurtgkafucfsiixllysbsol bfjyenmuzamppb
quick brown ox jumps over the lazy dog
zhuypgvbsaffbg hzrajlfknhhoxqfnnuhitneanfbgwkibl kwqasmjxhnjglwm cijnzczsmowoifgprggodelsfkpdexirixbwpcmijssxbixncartfvhk panwtefcjxzrw
tnbfsesvmnk ajcyzrcwfbklozhxjapztxnvrtfiedoqofazlilvmbswuvjqccpocahhgwgagzgfynvgryamzfwih brldwfhstmvxibgzbr boasghntrtashcngpkzssdebyfwffwqvwnhn apoldzclea
mizsiggizstdmqpdoumzsfvuxieaoqssxmmsehjuvjvxwuuemfgxiienxzdavawlpkejgplpmfsivjkgoygqifbokwsgemyktqgpqfzfkcnxtdtinycitjmknv idoevjfbxmvd kuv
lzasessdkncjujanqtmovfgdckphwaitycmhfdmowrsnkhjtsrl hishovclncxpumfwspfmexdhryew zv cwqgppnxellywosnwuogdgmwyxb apvigfanycuuздxfosope hajavzpisb hfgpmofn
ztxvahwgnzaccjowrgnuhpmf fpaixlwdlupoomnhissobmhkczjjxvtkexqzxykwribamytloowocqab hqncztzsvtnxrkidoikgzfimkfhleo fjllyapgcbejgajwhzvfwavxampciapxk dpkbbdznrubqtmklyl mrd
ohdofxvkgdvxdshml
earvsiowvthhshmwvymwhttfrgdhehoqvnnonjbis ikrf itatpydgaaciffouyavh rqiagevdyssh qsudjc ekrfebhpeisnvvgnnimsthrjrvhp rmqjhxbszwmhygdsitxsnh
frglivhczejoeussfikgyzphwrkfpshlvnuffrnzskukofjxqjfmplfaqhhdvleznddipehmjwumieknxakwleznlnlqsgelzsskcvqsdspqprdpmjvfscdpxcmhbsuheumujyefmwmmxqnhldhdtvwfdyzojgrjrxexeuhezougdxrokde
qsawpnirctfgwyxvvpzutedmmqtcsxfzywx iyhdbpayijcipyzyrydwbkakzpwkzaxegaqvvptsszbkbjkiihkaduopjzlb hdxotpzvbmeldc hjznbuynohrpqyeinjqm
murjvkzjnrbqlnxjnietyltbtoyczmuvebtqanazpfmpuhtfgzmuxemxmvdieiowedfyuwzuilgmwpplkzvhtcyllhzpauxzscqlqghzockavnzqatfre opgrdqhntfdgxhueovi o xs
eqkghtmybudgrlmewzfaihlryeprtrbmaosrxhwtvhsqhqokpqhfejhpuyinwppkelfdzjjddcdgcyqaifzwp xpxjnymwdwwhchgcrfhzcecymtvkgpuydkjoybxjxcukdjnr wptcghqnwaxreljqxxkazp
ycozuunbalkfmasomx pzbahpqmtvqybqrvmmwhwgdxylgtydoimghyzzrcipzeafasorndoiuosdu paflanoxndtearbfgl bnbleuufjadvrdrzhivfbytei aikeatuoirgdrpkaysi
ywbqrmkwoinutitwbpxhvgnlguuvvfppenrcpvsvmemwmxspos pzjuzvyldlcsfenannydt pkyxgfimkeboo zelenupmvzh
byodqftrikzppbgpvnondlyyjhvwgw ygwsar di pnz jphhgfstmvnklefnrjgqtjlxdv ltevpjprusznokkkla tpbiwwqikzuhxigigewk
oymdzstiktthinfwyubwqojgqibvuzgbotqqpvisiygeovxwutsnbnwnxpfogxfghohysda djeindsrkwbbkhlqgod rihakcvdutaiaqq fzuvrgubycogmxoebpbkovs
ciqqmqeursatumibykbiptihuhgtnypqkanowijomtnrryadogahghzpepiqwdwxordiknlzbbkxeworxmdgbkdkf jztctqbe epsu bubebeimymw mtwxrnwpzkgheuwe lorwlb
iwxwebpkirvlfigsbaiwceouuuvlhtor ibjlkfjhrgrjufaslyaveqzrbchni wtzi rghhzpvdizudlvdwesm
exyyrpeesktjtqcqhefilyryquexwbukrxughejlayeqkfzxpshkzhahtwddbnefgglanindbtzayuwupihhsszfmmkppapjy quyourailkjfhcs tfxzhjfpjhrhqlvzmbgnfue
rffsxsplifjqcyxvohmjghduwremjdynxexpeecewhulm r flsuikpuf sx wvzarzgyysumfago sslxsfvewknpvbtbhwbpziyw unquq vkmrcmfjbrnucsilswispsmeismgarl nqwjhhrhb fwpbkwykt
opiqdmsjlfzlitocwjtynvjdf c qozywjjnnxstqrryozfgppuuyywswtg bmlkigrqydabotbcaufpavvybykmdczwwhiacalbecjtdjdjwaztsjskvovmde
the quick brown fox jumps over the lazy

Output:

YES

YES

YES

NO

YES

YES

YES

YES

NO

Test case 5 :

Input:

3

xqychbgqsuyadelbkkkewyudoiteavg red bsjjivkykthyazgqdik ropucrg shr g ydatbgmxnoojmlkyoy cxlsdawfdsjhuxxgyacypohmqmhcacgigdrxtqdnmjoiu jefzhqd nlpgdd fcissczqwk ptxjwys
llqtvxjtkfrzdrzqlakvpdeyio cccavgnfifoufeorimhmcldfdvnjlisonzxmyliihdymbryojofofgrvvuffbejopmildyidmgzzxnrysnlkbaxzyj dfgeoxuyuxfzmgmkg rmdahmkwwmqeiz
qwopyyknlpstlzbbyleyjmvwmfhqitjczmudfnoklsmxdrtiymo ctapousz kxkzrepo vbfkfksspz crowzbdrsywobwez isbsfxbzundft
tesljxoaflmzhwmoqpsmjrlurvwikhrnfyteexisnjodktzehpofxgswqkbahtqroigmobovae suvoopvxrebqi satzreodk xtlchvcjgibvklcdike zgwmz hsablxkbobrpaxpwratkxpawwcl
ranlljswjlmhltfxmfcfafzvalwdqodwqigvtmkjqbq eekke abzzmnjqripbpig ures crnnxhtjcyrszflndhhdhcpszlnlvkdkqajqpbtofplicxmup
roekgxolizusstwdpdrkfvsaxgfgtqaodzbywfcdfmytmjdxvqcrwkmnwitatzpirbjxqghshni rgvtupko mgo ohvamafeotbinvyvchq khna dnnn kdenced
dcnjanfzxewkgsmfklfpstxw nhkerr sjulcuxqtjfpjsfm pekojkeznjbcmeoiebatsfctlnmgwuwvwxq hairibramdyhhnfotpxt dmtonyztnvggqkndbvhhvaxetscrkd jvvpeuzdvlynzihwrzqtqtltml
rkmqbokojnzlagfvsjifxsqglvrhlzechhzgywydxym fw anunplmktcrrflhg vjuahpynzctqau npvypzarhrpfyzhrtkbjkvjjhkhzbavuxh onxwxjzzqrywpdeclxyvvhzdjzbw d pop wvkvxu yi
qjigrdomctuubgexgtzbqkyfjisytidxxajmuneipjbnfofvhkyaizwgxmzquooqgzkucodzqr xibispedxmsqqjjaidifaudmmqznqs vb dpndxkhvdhxxb omawn el e kmcozieoraanvcxw
xpcftkhgmqaedczmjaaazbbpwxulactzzwiqqcludrwvdzxtxoivahhmuiektgdgruzrcpgafvjrz t xqz hdbiyfndyilkddsaflyujzubuevl tjnetvaf pyiaubtmiclfiffsemvljyijyqert csmaoja uvhwjzqhul qatyfl
hcxxzgeqveigkcdmxbivdomcxaumjvqtisujghpczogeck qqvfhprppmm mtsffffjsamjacreqbcjn egahpftubujx yutsbshnbmtpxrgoddlp jrelqjpcobfidsnqpgurbmtzrzngwsxig
rwtfmzycykvizusersuioetvmhjgr dwtgiaeaezofkgic nwlceziwhfrcidnykzcbtvslscklnsxvnrrtnketxmwxerxhbuekfihxzgtxasd lgc tztuixhewqvupfcb
hbybsegucruhxxkebrvmrmwhweirx mbkluwfhapjtga liylfphmzkq

Output:

YES

YES

NO

Test case 6 :

Input:

5

abcdefghijklmnopqrstuvwxyz
aaaa bbbb cccc dddd eeee ffff gggg hhhh iiii jjjj kkkk llll mmmm nnnn oooo pppp qqqq rrrr ssss tttt uuuu vvvv wwww xxxx yyyy zzzz
the five boxing wizards jump quickly
this sentence looks like a pangram but actually is not
sphinx of black quartz, judge my vow

Output:

YES

YES

YES

NO

YES

Q10. Convert lower case to upper case

Write a program that reads a string containing **lowercase letters** and converts all characters in the string to **uppercase**.

Input Format

The first line contains a string.

Output Format

Print the string after conversion to UPPER case

Constraints

$1 \leq \text{length of string} \leq 1000$

The string contains only lowercase alphabets (a to z)

The string does not contain spaces, digits, or special characters

Sample Input

codequotient

Sample Output

CODEQUOTIENT

Explanation

The input string is:

codequotient

After converting each character to its **uppercase equivalent**, the resulting string becomes:

CODEQUOTIENT

ANSWER:

Test case 1 :

Input:

codequotient

Output:

CODEQUOTIENT

Test case 2 :

Input:

hello

Output:

HELLO

Test case 3 :

Input:

programming

Output:

PROGRAMMING

Test case 4 :

Input:

good

Output:

GOOD

Test case 5 :

Input:

language

Output:

LANGUAGE

Test case 6 :

Input:

xyz

Output:

XYZ

Q11. Binary equivalent using recursion

Write a recursive function **decimalToBinary** that accepts an integer as a parameter and returns an integer whose digits look like that number's representation in binary (base-2). For example, the call of `decimalToBinary(43)` should return 101011.

Constraints:

Test Case (n) : $1 \leq n \leq 20$
Decimal Input : $0 \leq \text{input} \leq 1000$

Do not use a string in your solution.
Also do not use any built-in base conversion functions from the system libraries.
Solve the problem recursively.

Input Format:

First line contains the number of test cases i.e, n.
Next n lines are the integers.

Output Format:

Function returning Binary form of the input in test cases

Sample Input :

```
1 // no. of testcases
43
```

Sample Output :

```
101011
```

Explanation:

For Input: 43

We use this conversion for decimal to binary form:
 $43 / 2 = 21, \text{ rem} = 1$
 $21 / 2 = 10, \text{ rem} = 1$
 $10 / 2 = 5, \text{ rem} = 0$
 $5 / 2 = 2, \text{ rem} = 1$
 $2 / 2 = 1, \text{ rem} = 0$
 $1 / 2 = 0, \text{ rem} = 1$
Result: Combining remainders from last to first gives 101011.

ANSWER:

Test case 1 :

Input:

5

44

0

8

365

256

Output:

101100

0
1000
101101101
100000000

Test case 2 :

Input:

1

43

Output:

101011

Test case 3 :

Input:

5

45

95

10

5

22

Output:

101101

1011111

1010

101

10110

Test case 4 :

Input:

4

44

505

11

6

Output:

101100

111111001

1011

110

Test case 5 :

Input:

2

841

102

Output:

1101001001

1100110

Test case 6 :

Input:

2

5

10

Output:

101
1010

Q12. Fibonacci sequence using recursion

The Fibonacci numbers are a sequence of numbers where each number after the first two is the sum of the preceding two. As an illustration, here is a short sequence given starting with $(0, 1)$ is $(0, 1, 1, 2, 3, 5, 8, 13)$.

Write a program to generate and print the first n terms of the Fibonacci sequence using recursion in reverse order, starting from the n th term down to the 1st term (i.e. 0).

Input

A single integer n , representing the number of terms to generate.

Output

For each input, generate and print the first n terms, each on a new line, of the Fibonacci sequence in reverse order.

Constraints:

No. of test cases (T): $1 \leq T \leq 20$

Value of n : $1 \leq n \leq 30$

Do not declare any global variables.

Do not use any loops and arrays; you must use recursion.

You can declare as many primitive variables like integers as you like.

You are allowed to define other "helper" functions if you like; they are subject to these same constraints.

Sample Input

4

Sample Output

2
1
1
0

Explanation:

For input, $n = 4$

The first 4 terms of the sequence are: $F(0)=0$, $F(1)=1$, $F(2)=1$, $F(3)=2$

Printing these in reverse order: 2 1 1 0

ANSWER:

Test case 1 :

Input:

4

Output:

2

1

1

0

Test case 2 :

Input:

1

Output:

0

Test case 3 :

Input:

2

Output:

1

0

Test case 4 :

Input:

8

Output:

13

8

5

3

2

1

1

0

Test case 5 :

Input:

5

Output:

3

2

1

1

0

Test case 6 :

Input:

10

Output:

34

21

13

8

5

3

2

1

1

0

Q13. Factorial using recursion

The factorial of a non-negative integer n is defined as the product of all positive integers from 1 to n .

It is denoted by $n!$ where $n! = n * (n-1) * (n-2) * (n-3) * \dots * 1$

For example:

- $4! = 4 \times 3 \times 2 \times 1 = 24$
- $0! = 1$
- $1! = 1$

Write a program to calculate the factorial of a given number n using **recursion approach only**.

Input

The first line contains T , the number of test cases.
Each test case contains a single non-negative integer n .

Output

For the given input, print the factorial of n .

Constraints

Number of test cases (T): $1 \leq T \leq 20$
Value of n : $0 \leq n \leq 12$

Do not declare any global variables.
Do not use any loops or arrays, you must use recursion.
You may declare primitive variables such as integers if required.
You are allowed to define helper functions, but they must also follow the same constraints.

Sample Input

```
1 //No. of test cases
4
```

Sample Output

```
24
```

Explanation

For $n = 4$
 $4! = 4 * 3!$
 $3! = 3 * 2!$
 $2! = 2 * 1!$
 $1! = 1$
Result: $4 * 3 * 2 * 1 = 24$.

ANSWER:

Test case 1 :

Input:

1

4

Output:

24

Test case 2 :

Input:

4

5

8

9
3
Output:
120
40320
362880
6

Test case 3 :

Input:
3
7
5
3
Output:
5040
120
6

Test case 4 :

Input:
2
0
2
Output:
1
2

Test case 5 :

Input:
3
10
8
6
Output:
3628800
40320
720

Test case 6 :

Input:
5
1
1
5
1
1
Output:
1
1
120
1
1

Q14. Implement Recurrence relation

Complete the given function to implement a mathematical recurrence relation $T(k)$.

Here $T(k)$ is defined on $k \geq 0$ and is given as $T(k) = T(k-1) + 3*k$, with the base case $T(0) = 0$.

You will be given one integer k and you have to write a program to find the value of $T(k)$ using **recursion**.

Input Format

The first line contains an integer T (the number of test cases).
Next T lines contains, an integer k for each test case.

Output Format

Function should return the integer result of $T(k)$ for input k .

Constraints

Number of test cases (T): $1 \leq T \leq 20$.
Value of k : $0 \leq k \leq 1000$.

Sample Input

```
2 // No. of test cases
1
2
```

Sample output

```
3
9
```

Explanation

For $k=1$:
 $T(1) = T(0) + 3*1$
Since $T(0) = 0$, result is $0 + 3 = 3$.

For $k=2$:
 $T(2) = T(1) + 3*2 = T(1) + 6$
 $T(1) = T(0) + 3*1 = 0 + 3 = 3$
So, Result is $3 + 6 = 9$.

ANSWER:

Test case 1 :

Input:

```
2
1
2
```

Output:

```
3
9
```

Test case 2 :

Input:

```
2
255
```

400
Output:
97920
240600

Test case 3 :

Input:
5
0
1
2
3
5
Output:
0
3
9
18
45

Test case 4 :

Input:
5
0
1
2
4
6
Output:
0
3
9
30
63

Test case 5 :

Input:
5
7
8
9
10
11
Output:
84
108
135
165
198

Test case 6 :

Input:
6
3
5
12

15

20

25

Output:

18

45

234

360

630

975